



Fundusze
Europejskie
Wiedza Edukacja Rozwój



Rzeczpospolita
Polska

Politechnika
Warszawska

Unia Europejska
Europejski Fundusz Społeczny



Algorytmy w Inżynierii Danych

Wyznaczanie wartości własnych

Bartosz Chaber

Macierz ortogonalna

Wróćmy do definicji z poprzednich wykładów.

Macierz ortogonalna to macierz *unitarna* o wartościach rzeczywistych. Iloczyn macierzy ortogonalnej i jej *sprzężenia hermitowskiego* (w naszym przypadku *transpozycji*) jest macierzą *jednostkową*.

Przykładem macierzy ortogonalnej jest np. macierz obrotu. Na macierz ortogonalną możemy patrzeć jak na zbiór *ortogonalnych* wektorów kolumnowych.

```
> Q = [cos(π/4) -sin(π/4)
        sin(π/4)  cos(π/4)]
2×2 Matrix{Float64}:
 0.707107 -0.707107
 0.707107  0.707107
> Q * Q'
2×2 Matrix{Float64}:
 1.0  0.0
 0.0  1.0
```

Macierz podobna

W metodach opisanych poniżej wykorzystana będzie własność macierzy podobnych, czyli takich, że widmo macierzy A jest takie samo jak macierzy $B^{-1} \cdot A \cdot B$, gdy B jest **odwracalną** macierzą.

Pewną trudnością może być wyznaczenie macierzy odwrotnej do B . Jednak warto pamiętać o macierzach ortogonalnych, dla których prawdziwa jest zależność: $Q^{-1} = Q^T$.

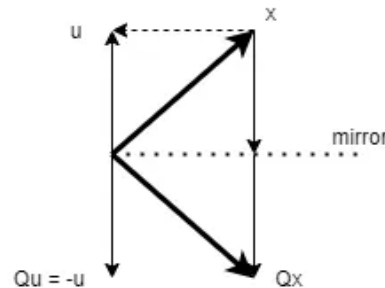
Macierz Hessenberga

Macierz, *prawie* górnotrójkątna. Posiada niezerowe elementy również *pod* główną przekątną.

```
> using LinearAlgebra
> hessenberg(rand(6,6)).H
6×6 UpperHessenberg{Float64, Matrix{Float64}}:
 0.319084 -1.08695  0.806435  0.162724 -0.143383  0.0952936
-1.01479  1.26041 -1.18598  0.224861  0.313971  0.472413
.         -0.940752  0.824951 -0.341308 -0.632469 -0.181593
.          .        -0.689652  0.403017  0.395576  0.398501
.          .          .        0.24755  0.0145165  0.0577847
.          .          .          .        0.0482109 -0.189054
```

Transformacja Householdera

Jedną z metod pozwalających znaleźć postać Hessenberga pełnej macierzy A o wymiarach $N \times N$ jest metoda odbić Householdera. Aby zrozumieć tę metodę warto patrzeć na macierz jako na przestrzeń N -wymiarowych wektorów. Wyzerowanie niektórych składowych tych wektorów można dokonać poprzez odbicie w N -wymiarowej przestrzeni.



```

function householder_hessenberg(A :: Matrix{Float64})
    H = copy(A)
    N = size(A, 1)

    for i=1:N-2
         $\alpha$  = H[i+1:N, i] # FIXME
         $\sigma$  = norm( $\alpha$ )
         $\beta$  = 1.0 / ( $\sigma$  * ( $\sigma$  + abs( $\alpha$ [1])))
        expj $\phi$  =  $\alpha$ [1] < 0.0 ? -1.0 : 1.0
        u = copy( $\alpha$ )
        u[1] = expj $\phi$  * ( $\sigma$  + abs( $\alpha$ [1]))
        T = diagm(ones(N))
        T[i+1:N, i+1:N] .= diagm(ones(length( $\alpha$ ))) -  $\beta$  * u * u'
        H .= T * H * T
    end

    return H
end

```

W powyższym algorytmie warto zwrócić uwagę na wykorzystanie macierzy odbicia T . Ponieważ odbicie względem wybranej hiperpłaszczyzny jest symetryczne, wygląda na to, że $T^{-1} = T$. Co za tym idzie, macierz $T \cdot A \cdot T$ będzie miała takie samo widmo jak macierz A . Docelowo, po pewnej liczbie odbić otrzymamy macierz równoważną macierzy A , ale w postaci Hessenberga.

Rozkład QR macierzy

Poprzez delikatną modyfikację kodu możliwe jest też dokonanie tzw. dekompozycji QR “rozbijającej” macierz A na iloczyn macierzy $Q \cdot R$. Macierz Q jest macierzą ortogonalną, natomiast macierz R jest macierzą górnotrójkątną.


```

function householder_qr(A :: Matrix{Float64})
    R = copy(A)
    N = size(A, 1)
    Q = diagm(ones(N))
    for i=1:N-1
         $\alpha$  = R[i:N, i]
         $\sigma$  = norm( $\alpha$ )
         $\beta$  = 1.0 / ( $\sigma$  * ( $\sigma$  + abs( $\alpha$ [1])))
        expj $\phi$  =  $\alpha$ [1] < 0.0 ? -1.0 : 1.0
        u = copy( $\alpha$ )
        u[1] = expj $\phi$  * ( $\sigma$  + abs( $\alpha$ [1]))
        T = diagm(ones(N))
        T[i:N, i:N] .= diagm(ones(length( $\alpha$ ))) -  $\beta$  * u * u'
        R = T * R # te linie zostały zmienione
        Q = Q * T # względem metody Householdera zdefiniowanej wcześniej
    end

    return Q, R
end

```

```
> using SparseArrays
> dropalmostzeros(A, ε) = sparse([abs(aij) > ε ? aij : 0.0 for aij in A])
> A = rand(5, 5);
> Q, R = householder_qr(A);
```

```
> dropalmostzeros(Q' * Q, 1e-15)
```

5×5 SparseMatrixCSC{Float64, Int64} with 5 stored entries:

```
1.0  .  .  .  .
.  1.0  .  .  .
.  .  1.0  .  .
.  .  .  1.0  .
.  .  .  .  1.0
```

```
> dropalmostzeros(R, 1e-15)
```

5×5 SparseMatrixCSC{Float64, Int64} with 15 stored entries:

```
-1.24317 -0.881957 -1.16746 -0.739276 -1.40247
.      -1.12885  -0.146022 -0.00999683 -0.812501
.      .      -0.45926   0.177693  -0.418866
.      .      .      -0.676806  -0.222057
.      .      .      .      0.22363
```

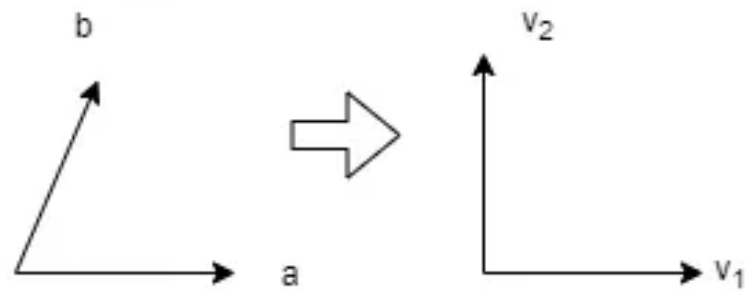
Metoda Grama-Schmidta

Alternatywną metodą do wyznaczenia rozkładu QR wybranej macierzy jest A jest metoda Grama-Schmidta. Wykorzystuje ona tę samą interpretację macierzy, co metoda odbić Householdera, ale zamiast odbić rzutuje ona kolejne kierunki w celu uformowania ortogonalnej bazy Q . Macierz R otrzymuje się potem poprzez rzutowanie macierzy A na kierunki Q .

$$Q \cdot R = A \quad | \cdot Q^{-1}$$

$$Q^{-1} \cdot Q \cdot R = Q^{-1} \cdot A$$

$$R = Q^{-1} \cdot A = Q^T \cdot A$$



Podstawową operacją w metodzie Grama-Schmidta jest rzutowanie wektora $\boldsymbol{v}$ na wektor $\boldsymbol{u}$.

$$\text{proj}_{\boldsymbol{u}} \boldsymbol{v} = \frac{\boldsymbol{u} \cdot \boldsymbol{v}}{\boldsymbol{u} \cdot \boldsymbol{u}} \boldsymbol{u}$$

Jeżeli zarówno $\boldsymbol{u}$ jak i $\boldsymbol{v}$ są wersorami, tj. mają jednostkową długość to wzór uprości się do:

$$\text{proj}_{\boldsymbol{u}} \boldsymbol{v} = (\boldsymbol{u} \cdot \boldsymbol{v}) \boldsymbol{u}$$

Klasyczna wersja algorytmu:

```
function cgs(A :: Matrix{Float64})  
    n, _ = size(A)  
    v = A[:,1]  
    Q = zeros(n, n)  
  
    for j = 1:n  
        v = A[:,j]  
  
        for k = 1:j-1  
            v -= (Q[:,k] · A[:,j]) * Q[:,k]  
        end  
  
        Q[:,j] .= v ./ norm(v)  
    end  
    R = Q' * A  
    return Q, R  
end
```

Zmodyfikowana metoda Grama-Schmidta

W klasycznej wersji, wektor u powstaje poprzez odejmowanie rzutów v na poprzednie kierunki:

$$u_i = v - \text{proj}_{u_1} v - \text{proj}_{u_2} v - \dots$$

Takie sformułowanie w arytmetyce zmiennoprzecinkowej powoduje dość szybką utratę ortogonalności między kolejnymi kierunkami.

Równoważne, ale bardziej stabilne numeryczne sformułowanie można zapisać następująco:

$$\mathbf{u}_i^{(1)} = \mathbf{v} - \text{proj}_{\mathbf{u}_1} \mathbf{v}$$

$$\mathbf{u}_i^{(2)} = \mathbf{u}_i^{(1)} - \text{proj}_{\mathbf{u}_2} \mathbf{u}_i^{(1)}$$

$$\mathbf{u}_i^{(3)} = \mathbf{u}_i^{(2)} - \text{proj}_{\mathbf{u}_3} \mathbf{u}_i^{(2)}$$

...

$$\mathbf{u}_i^{(i-1)} = \mathbf{u}_i^{(i-2)} - \text{proj}_{\mathbf{u}_{i-1}} \mathbf{u}_i^{(i-2)}$$

Zmodyfikowana wersja algorytmu:

```
function mgs(A :: Matrix{Float64})  
    n, _ = size(A)  
    v = A[:,1]  
    Q = zeros(n, n)  
  
    for j = 1:n  
        v = A[:,j]  
  
        for k = 1:j-1  
            v -= (Q[:,k] · v) * Q[:,k]  
        end  
  
        Q[:,j] = v ./ norm(v)  
    end  
    R = Q' * A  
    return Q, R  
end
```



```
> A = rand(200, 200)
> Q, _ = cgs(A); @show norm(I - Q' * Q);
norm(I - Q' * Q) = 1.2391108200049796e-11
> Q, _ = mgs(A); @show norm(I - Q' * Q);
norm(I - Q' * Q) = 1.8589603619760764e-13

> A = [(i == j ? 1.0001 : 1)/(i+j-1) for i=1:200, j=1:200];
> Q, _ = cgs(A); @show norm(I - Q' * Q);
norm(I - Q' * Q) = 179.8030660935869
> Q, _ = mgs(A); @show norm(I - Q' * Q);
norm(I - Q' * Q) = 4.662298863101829e-10
```

Algorytm Francisa

Jednym z najważniejszych algorytmów ostatnich lat jest **algorytm QR**, który pozwala na szybkie wyznaczenie wartości własnych macierzy.

Aby nie mylić go z *dekompozycją QR* nazwiemy go od nazwiska oryginalnego autora: Johna Francisa (równolegle i niezależnie opracowała go też Vera N. Kublanovskaya).

Najprostsza implementacja wykorzystuje rozkład QR macierzy, a następnie wymnożenie składników “na odwrót”. Można pokazać, że wynik takiego mnożenia ma to samo widmo, co macierz podlegająca rozkładowi, ponieważ:

$$Q \cdot R = A \mid \cdot Q \text{ (prawostronnie)}$$

$$Q \cdot R \cdot Q = A \cdot Q \mid \cdot Q^{-1} \text{ (lewostronnie)}$$

$$Q^{-1} \cdot Q \cdot R \cdot Q = Q^{-1} \cdot A \cdot Q.$$

Ostatecznie sprowadzając nas do:

$$R \cdot Q = Q^{-1} \cdot A \cdot Q.$$

Co zgodnie z własnością macierzy podobnych pozwala nam stwierdzić, że widma $\mathbf{R} \cdot \mathbf{Q}$ jest takie samo jak macierzy $\mathbf{A}$. Jednak zaletą takiej transformacji jest to, że po pewnej liczbie iteracji wynikowa macierz staje się macierzą górnątrójkątną, której wartości własne znajdują się na głównej diagonalu.

```
function explicitfrancis(A :: Matrix{Float64},  
maxiter)  
    M = copy(A)  
    N = size(A, 1)  
    λs = zeros(N)  
    for k = 1:maxiter  
        Q, R = grammschmidt(M)  
        mul!(M, R, Q)  
        λs .= diag(M)  
    end  
  
    return λs  
end
```

```
> A = randn(5,5)  
> A *= A' # niech A będzie  
symetryczna  
> λ = explicitfrancis(A, 100)  
[21.87917065432922,  
 14.060237996936197,  
  2.3115851771234817,  
  0.6475979594944075,  
  0.06021804947597513]  
  
> λ = reverse(eigvals(A))  
[21.879170654329148,  
 14.060237996936166,  
  2.311585177123479,  
  0.6475979594944056,  
  0.06021804947597582]
```

Ulepszony algorytm Francisa

Ciekawą wersją algorytmu Francisa jest wykorzystanie własności postaci Hessenberga. Otóż postać Hessenberga o wymiarach $n \times n$ wymaga $n - 1$ operacji do przekształcenia tej macierzy do postaci górnotrójkątnej.

Eliminacji niechcianych elementów można dokonać poprzez wykorzystanie *obrotów Givensa*, pozwalających na wyzerowanie dowolnego elementu macierzy.

Obroty Givensa

Macierz G pozwala na “obrócenie” wektorów w kolumnach A tak, aby wybrany element macierzy $A_{i,j}$ był równy zero:

```
using LinearAlgebra
function givensrot(A, i, j)
    q = sqrt(A[j,j]^2 + A[i,j]^2);
    c = A[j,j] / q;
    s = A[i,j] / q;

    N = size(A, 1);
    G = diagm(ones(N));
    G[i,i] = c; G[j,j] = c;
    G[j,i] = s; G[i,j] = -s;

    return G
end
```

```
> A = rand(3,3)
> G = givensrot(A, 2, 1)
3×3 Matrix{Float64}:
 0.348304  0.937382  0.0
-0.937382  0.348304  0.0
 0.0       0.0       1.0
> G * A
3×3 Matrix{Float64}:
 0.997136      1.00958      0.328816
(5.551e-17)    0.13537     -0.787595
 0.422237      0.23822      0.083311
```

Zgodnie z twierdzeniem w [5] złożenie $n - 1$ macierzy Givensa pozwalających na wyeliminowanie pod-diagonali jest **równoważne** z przeprowadzeniem rozkładu QR tej macierzy. Następnie, można dokończyć przekształcenie przez podobieństwo, aplikując sekwencję macierzy Givensa z prawej strony.

Theorem 2.2.6 Suppose $A \in \mathbb{C}^{n \times n}$ is a Hessenberg matrix. Let H_i be generated as follows $H_1 = A$

$$H_{i+1} = G_i^T H_i, \quad i = 1, \dots, n-1$$

where $G_i = G(i, i+1, (H_i)_{i,i}/r_i, (H_i)_{i+1,i}/r_i)$ and $r_i = \sqrt{(H_i)_{i,i}^2 + (H_i)_{i+1,i}^2}$ and we assume $r_i \neq 0$. Then, $H_n = R$ is upper triangular and

$$A = (G_1 G_2 \dots G_{n-1}) H_n = QR$$

is a QR-factorization of A .

$$A_{k+1} = RQ = Q^* A_k Q = H_n G_1 \dots G_{n-1}$$

The application of $G_1 \dots G_{n-1}$ can also be done in $\mathcal{O}$ and consequently

the complexity of one Hessenberg QR step = $\mathcal{O}(n^2)$

The algorithm is summarized in Algorithm 2.2.2. In the algorithm $[c, s] = \text{givens}(a, b)$ returns $c = a/\sqrt{a^2 + b^2}$ and $s = b/\sqrt{a^2 + b^2}$.

Algorithm 3 Hessenberg QR algorithm

Input: A Hessenberg matrix $A \in \mathbb{C}^{n \times n}$

Output: Upper triangular T such that $A = UTU^*$ for an orthogonal matrix U .

Set $A_0 := A$

for $k = 1, \dots$ **do**

 // One Hessenberg QR step

$H = A_{k-1}$

for $i = 1, \dots, n-1$ **do**

$[c_i, s_i] = \text{givens}(h_{i,i}, h_{i+1,i})$

$$H_{i:i+1,i:n} = \begin{bmatrix} c_i & s_i \\ -s_i & c_i \end{bmatrix} H_{i:i+1,i:n}$$

end for

for $i = 1, \dots, n-1$ **do**

$$H_{1:i+1,i:i+1} = H_{1:i+1,i:i+1} \begin{bmatrix} c_i & -s_i \\ s_i & c_i \end{bmatrix}$$

end for

$A_k = H$

end for

Return $T = A_\infty$

```
> A1 = A = rand(3,3)
> G1 = givensrot(A1, 2, 1); A2 = G1 * A1
> G2 = givensrot(A2, 3, 1); A3 = G2 * A2
> G3 = givensrot(A3, 3, 2); A4 = G3 * A3
> dropalmostzeros(A4, 1e-15)
0.934422  0.936767  0.377592
.          0.583307  0.853773
.          .        -0.190757
```

```
> Q = (G1' * G2' * G3')
> dropalmostzeros(Q' * Q, 1e-15)
1.0      .      .
.      1.0      .
.      .      1.0
```

```
> R = (G3 * (G2 * (G1 * A)))
> dropalmostzeros(R, 1e-15)
0.934422  0.936767  0.377592
.          0.583307  0.853773
.          .        -0.190757
```

Algorytm rozkładu QR wykorzystujący obroty Givensa

```
function givens_qr(A)
    N = size(A, 1)
    Q = diagm(ones(N))
    R = copy(A)
    for i=1:N-1
        for j=i+1:N
            G = givensrot(R, j, i)
            Q = Q * G'
            R = G * R
        end
    end
    return Q, R
end
```

```
function francis(A :: Matrix{Float64}, maxiter)
    H = householder_hessenberg(A)
    N = size(H, 1)
    G = [[0. 0.; 0. 0.] for j=1:N-1]
    λs = zeros(N)
    for k = 1:maxiter
        for j = 1:N-1
            q = sqrt(H[j,j]^2 + H[j+1,j]^2);
            c = H[j, j] / q;
            s = H[j+1,j] / q;
            G[j] = [c -s; +s c]
            H[j:j+1, j:N] = G[j]' * H[j:j+1, j:N] # => 2 x (N-j)
        end
        for j = 1:N-1
            H[1:j+1, j:j+1] = H[1:j+1, j:j+1] * G[j] # (j) x 2
        end
        λs .= diag(H)
    end
    return λs
end
```

Teraz przyspieszymy ten algorytm wykorzystując statyczne macierze...

```

using StaticArrays
function francis(A :: Matrix{Float64},
maxiter)
    H = householder_hessenberg(A)
    N = size(H, 1)
    G = [@SMatrix [0. 0.; 0. 0.] for j=1:N-1]
    λs = zeros(N)
    for k = 1:maxiter
        for j = 1:N-1
            q = sqrt(H[j,j]^2 + H[j+1,j]^2);
            c = H[j, j] / q;
            s = H[j+1,j] / q;
            Gj = G[j] = @SMatrix [c -s; +s c]
            # ...
        end
        # ...
        for k=j:N
            H[j, k] = Gj[1,1] * H[j, k] + Gj[2,1] * H[j+1, k]
            H[j+1,k] = Gj[1,2] * H[j, k] + Gj[2,2] * H[j+1, k]
        end
    end
    for j = 1:N-1
        Gj = G[j]
        for k=1:j+1
            H[k,j] = H[k,j] * Gj[1,1] + H[k,j+1] * Gj[2,1]
            H[k,j+1] = H[k,j] * Gj[1,2] + H[k,j+1] * Gj[2,2]
        end
    end
    λs .= diag(H)
end
return λs
end

```

Dynamiczna alokacja macierzy obrotu na sterpie:

```
> using BenchmarkTools  
> @btime francis(rand(5,5), 100);  
298.781 μs (4139 allocations: 283.61 KiB)
```

Dynamiczna alokacja macierzy obrotu na stercie:

```
> using BenchmarkTools  
> @btime francis(rand(5,5), 100);  
298.781 μs (4139 allocations: 283.61 KiB)
```

Statyczna alokacja macierzy obrotu na stosie:

```
> using BenchmarkTools  
> @btime francis(rand(5,5), 100);  
19.182 μs (131 allocations: 14.41 KiB)
```

Literatura

- [1] Tony Driscoll, Krylov Subspaces MATH 426 / CISC 410, 2012, url: https://www.youtube.com/watch?v=ji__O4deIZo, dostę: 29.05.2023
- [2] Nathan Kutz, Applied Linear Algebra GMRES, 2020, url: <https://www.youtube.com/watch?v=pPoLsRkBha4>, dostę: 29.05.2023
- [3] Greg Fasshauer, Chapter 14 - Arnoldi Iteration and GMRES, url: http://www.math.iit.edu/~fass/477577_Chapter_14.pdf, dostę: 29.05.2023
- [4] Tom Trogdon, Fall 2016: UCI MATH 105A: Numerical Analysis, url: <https://www.math.uci.edu/~ttrogdon/105A/html/Lecture23.html>, dostę: 30.05.2023

- [5] Elias Jarlebring, Lecture Notes in Numerical Linear Algebra, Chapter 2: QR algorithm, page 9, url: <https://people.kth.se/~eliasj/qrmethod.pdf>

Dziękuję za uwagę